

[illegible]

1. **Comprender** los fundamentos de las proposiciones lógicas y su representación a través de tablas de verdad, estableciendo la base del razonamiento computacional.
2. **Diseñar** y analizar circuitos lógicos utilizando compuertas básicas y derivadas, aplicándolos a situaciones reales del entorno local y tecnológico.
3. **Simplificar** expresiones booleanas complejas mediante axiomas y teoremas matemáticos, optimizando procesos de toma de decisiones en el desarrollo de software e infraestructura de hardware.



PRÁCTICA



¿Cómo automatizamos y protegemos los recursos de nuestro CEA sin depender de la intervención humana constante?

Imagina la siguiente situación en el nuevo y equipado módulo educativo del CEA Herman Ortiz Camargo, ubicado en el Barrio Saó. El centro acaba de adquirir equipos de última generación para la sala de computación. El director necesita un sistema de alarma y control de iluminación inteligente que opere bajo ciertas condiciones estrictas para ahorrar energía y garantizar la seguridad.

Para resolver este desafío, se te encomienda a ti, como futuro Técnico en Sistemas Informáticos, diseñar la lógica del sistema. Las reglas que te han dado son las siguientes:

1. Las luces del pasillo principal del CEA solo deben encenderse si es de noche **y** el sensor detecta movimiento en la entrada que da hacia el Barrio Saó.
2. La alarma de seguridad del laboratorio de computación debe activarse de inmediato si se abre la puerta principal **o** si se rompe la ventana trasera que colinda con la calle principal.
3. El sistema de riego automático para los jardines del recinto debe funcionar **solo si no** está lloviendo en la zona de Pailón.

Adicionalmente, se planea expandir esta lógica al Barrio 24 de Septiembre para controlar el acceso a la estación de tren, y al Barrio 27 de Mayo para automatizar las luces del coliseo municipal.

Antes de tocar un solo cable o escribir una línea de código, necesitas una forma de traducir estas reglas humanas (condiciones) a un lenguaje que las computadoras y los microcontroladores puedan procesar sin margen de error. ¿Cómo representarías estas condiciones matemáticamente para que un sistema electrónico tome la decisión correcta en fracciones de segundo?



TEORÍA



2.1. PROPOSICIONES LÓGICAS Y CONSTRUCCIÓN DE TABLAS DE VERDAD

En el mundo de la informática, las computadoras no entienden de ambigüedades; operan en un universo estrictamente binario. Todo se reduce a estados de encendido o apagado, verdadero o falso, 1 o 0. Para modelar esto, utilizamos las **proposiciones lógicas**.

Una proposición lógica es una afirmación que puede ser evaluada de forma inequívoca como Verdadera (V) o Falsa (F), pero nunca ambas al mismo tiempo.

1. *Ejemplo de proposición válida:* "El sensor de movimiento del Barrio Central Fátima está activado" (Puede ser V o F).
2. *Ejemplo no válido:* "¿A qué hora abre el comercio formal?" (Es una pregunta, no tiene valor de verdad).

Para visualizar todas las posibles combinaciones de estas proposiciones, los Técnicos en Sistemas Informáticos utilizamos las **Tablas de Verdad**. Una tabla de verdad desglosa todas las combinaciones de estados de las entradas y muestra cuál será el resultado (salida) de la operación lógica. La cantidad de combinaciones posibles se calcula con la fórmula 2^n , donde n es el número de variables (sensores, interruptores, proposiciones).

Si tenemos dos sensores (A y B) monitoreando el parque infantil del Barrio Residencial Norte, tendremos $2^2 = 4$ combinaciones posibles.

Sensor A	Sensor B
0	0
0	1
1	0
1	1

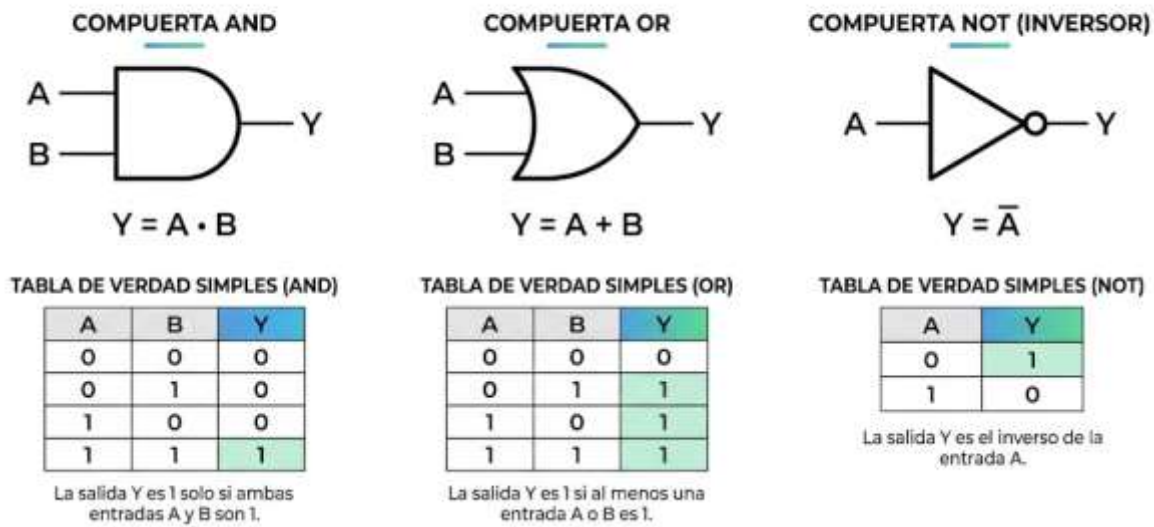
(Nota: En lógica digital, 0 representa Falso/Apagado y 1 representa Verdadero/Encendido).



2.2. COMPUERTAS LÓGICAS BÁSICAS (AND, OR, NOT)

Las operaciones que realizamos con las proposiciones se materializan en hardware a través de circuitos electrónicos llamados **compuertas lógicas**. Estas son el bloque de construcción fundamental de cualquier procesador.

SÍMBOLOS ESQUEMÁTICOS Y TABLAS DE VERDAD DE COMPUERTAS LÓGICAS BÁSICAS (NORMA ANSI/IEEE)



Compuerta AND (Y lógica):

Esta compuerta requiere que **todas** sus entradas sean verdaderas para que la salida sea verdadera. Equivale a la multiplicación binaria ($Y = A \cdot B$).

1. *Caso de uso en el CEA:* La alarma del laboratorio (Y) se activa SOLO SI el sistema está armado (A) AND se detecta una intrusión (B). Si el sistema está armado pero no hay intrusión, la alarma no suena.

Compuerta OR (O lógica):

Esta compuerta produce una salida verdadera si **al menos una** de sus entradas es verdadera. Equivale a la suma binaria lógica ($Y = A + B$).

1. *Caso de uso en Pailón:* Una bomba de agua en la plaza de San Expedito (Y) se enciende si el nivel del tanque principal es bajo (A) OR si el nivel del tanque secundario es bajo (B). Con que uno esté vacío, la bomba trabaja.

**Compuerta NOT (Inversor):**

A diferencia de las anteriores, esta compuerta solo tiene una entrada. Su función es invertir el estado lógico: si entra un 1, sale un 0; si entra un 0, sale un 1 ($Y = \overline{A}$).

1. *Caso de uso en Pailón:* Las luces de la iglesia del Barrio Central Fátima (Y) se encienden cuando NOT hay luz solar (A).

2.3. COMPUERTAS DERIVADAS Y FUNCIONES LÓGICAS COMPLEJAS (NAND, NOR, XOR)

A partir de las básicas, la electrónica digital nos ofrece compuertas derivadas que permiten optimizar y reducir la cantidad de componentes necesarios.

Compuerta NAND (NOT AND):

Es la inversa de la compuerta AND. Su salida es falsa *solo* cuando todas sus entradas son verdaderas ($Y = \overline{A \cdot B}$). Es conocida como una "compuerta universal" porque se puede construir cualquier circuito lógico utilizando únicamente compuertas NAND.

Compuerta NOR (NOT OR):

Es la inversa de la compuerta OR. Su salida es verdadera *solo* cuando todas sus entradas son falsas ($Y = \overline{A + B}$).

1. *Caso de uso en Las Misiones:* Un sistema de enfriamiento de servidores se mantiene inactivo (1) SOLO SI la temperatura no es alta (0) NOR hay exceso de carga de procesamiento (0).

Compuerta XOR (OR Exclusiva):

Esta compuerta es fundamental para realizar sumas binarias dentro del procesador. Su salida es verdadera si y solo si las entradas son *diferentes* entre sí ($Y = A \oplus B$). Si ambas son 0 o ambas son 1, el resultado es 0.

1. *Caso de uso:* El control de iluminación de un pasillo del módulo del CEA con dos interruptores (uno al principio y otro al final). Si cambias el estado de *cualquiera* de los dos, la luz cambia de estado.



COMPUERTA XOR Y SU EQUIVALENTE CON COMPUERTAS BÁSICAS (AND, OR, NOT)

SÍMBOLO DE COMPUERTA XOR (ANSI/IEEE)

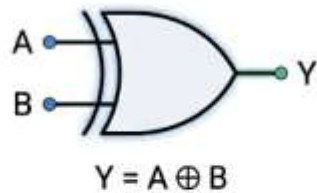


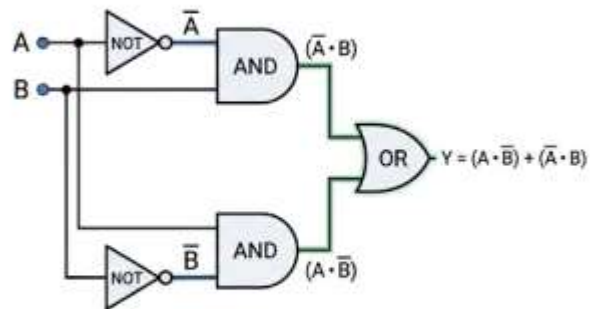
TABLA DE VERDAD Y COMPROBACIÓN DE EQUIVALENCIA

A	B	$\bar{A}$	$\bar{B}$	$(A \cdot \bar{B})$	$(\bar{A} \cdot B)$	XOR $Y = A \oplus B$	EQUIVALENTE $Y = (A \cdot \bar{B}) + (\bar{A} \cdot B)$
0	0	1	1	0	0	0	0
0	1	1	0	0	1	1	1
1	0	0	1	1	0	1	1
1	1	0	0	0	0	0	0

Ambos circuitos tienen la misma tabla de verdad, demostrando que son lógicamente equivalentes. La compuerta XOR es una abstracción útil y compacta.

CIRCUITO EQUIVALENTE CON COMPUERTAS BÁSICAS

$$Y = (A \cdot \bar{B}) + (\bar{A} \cdot B)$$



2.4. AXIOMAS, TEOREMAS Y PROPIEDADES DEL ÁLGEBRA DE BOOLE

George Boole desarrolló un sistema matemático en 1854 que hoy es el pilar de la informática. El Álgebra de Boole nos permite tratar expresiones lógicas con el rigor de las ecuaciones algebraicas. En tu carrera como Técnico en Sistemas Informáticos, usarás estas leyes para simplificar lógica en el código o en hardware.

Propiedades fundamentales:

1. **Conmutativa:** El orden de los factores no altera el producto.
 1. $A \cdot B = B \cdot A$
 2. $A + B = B + A$
2. **Asociativa:** Permite agrupar variables sin alterar el resultado final.
 1. $A \cdot (B \cdot C) = (A \cdot B) \cdot C$
3. **Distributiva:** Un elemento se distribuye sobre un grupo.
 1. $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$
 2. $A + (B \cdot C) = (A + B) \cdot (A + C)$ (¡Esta es exclusiva del álgebra de Boole!)

**Teoremas de De Morgan:**

Estos teoremas son esenciales para transformar puertas AND en OR (y viceversa) negando las variables. Son la herramienta número uno de cualquier Técnico para optimizar circuitos.

1. **Primer Teorema:** La negación de un producto lógico es igual a la suma lógica de sus negaciones.

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

2. **Segundo Teorema:** La negación de una suma lógica es igual al producto lógico de sus negaciones.

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

2.5. SIMPLIFICACIÓN DE EXPRESIONES BOOLEANAS

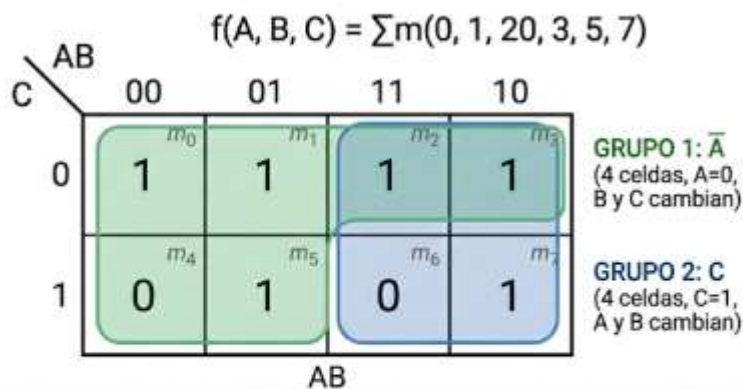
Cuando un sistema crece, como integrar las luces, bombas y alarmas de los barrios Oto Waver y Ovidio Barberi en un solo panel de control en el CEA, la expresión matemática resultante puede ser enorme. Implementar eso sin simplificar cuesta más dinero, usa más energía y procesadores más caros.

La simplificación busca encontrar la expresión mínima equivalente. Existen dos métodos principales:

1. **Método Algebraico:** Consiste en aplicar los teoremas y propiedades del Álgebra de Boole (como sacar factor común lógico) paso a paso hasta reducir la expresión a su mínima forma.
2. **Mapas de Karnaugh (Método Gráfico):** Es una herramienta visual que acomoda las tablas de verdad en una cuadrícula bidimensional. Permite agrupar los "1" lógicos adyacentes (en grupos de 2, 4, 8) para cancelar visualmente las variables que cambian de estado, obteniendo la ecuación simplificada de manera rápida e intuitiva.



MAPA DE KARNAUGH DE 3 VARIABLES Y SIMPLIFICACIÓN (EJEMPLO RESUELTO)



REGLAS DE AGRUPACIÓN CLAVE

- Solo agrupar '1's.
- Agrupar solo en potencias de 2 (1, 2, 4, 8).
- Maximizar el tamaño del grupo.
- Superponer grupos está permitido.

TABLA DE VERDAD

A	B	C	$\bar{A}$	C	f
0	0	0	1	0	0
0	1	1	1	0	1
0	1	0	0	1	0
1	1	0	1	1	1
0	0	1	0	0	0
1	0	1	1	0	1
7	1	0	1	1	1
8	1	0	0	1	0

PROCESO DE SIMPLIFICACIÓN DE LA FUNCIÓN

- Identificación de Términos (Grupos):
 - Grupo Verde: $\bar{A}$ (A=0 se mantiene, B y C cambian)
 - Grupo Azul: C (C=1 se mantiene, A y B cambian)
- Combinación (OR):
- Función Final: $f(A, B, C) = \bar{A} + C$

2.6. APLICACIÓN DE LA LÓGICA BOOLEANA EN LA DOMÓTICA Y SISTEMAS IOT CON MICROCONTROLADORES

Hoy en día, el Álgebra de Boole no se queda en el papel; salta a la realidad física a través del Internet de las Cosas (IoT). Microcontroladores accesibles como el ESP32, combinados con módulos de relés de 8 canales, son utilizados frecuentemente por los Técnicos para crear domótica.

Al programar el firmware de estos dispositivos, las condiciones lógicas (AND, OR) determinan en qué momento exacto el microcontrolador debe enviar la señal digital (1) al relé magnético para cerrar el circuito de alta tensión y encender, por ejemplo, el sistema de ventilación de los campos deportivos en el Barrio 24 de Septiembre. Sin una lógica booleana impecable, los relés actuarían de manera errática, causando posibles cortocircuitos o fallas en el servicio.

2.7. OPERACIONES BIT A BIT (BITWISE) EN LA PROGRAMACIÓN DE SOFTWARE

En el desarrollo Full-Stack, especialmente trabajando con lenguajes como Python, JavaScript o al configurar permisos en bases de datos PostgreSQL, la lógica booleana se aplica directamente en la gestión de memoria. Las operaciones *Bitwise* permiten manipular datos a nivel binario.



Por ejemplo, si un Técnico asigna roles de usuario (Lectura, Escritura, Ejecución) en una plataforma educativa del CEA, puede usar operaciones AND bit a bit para verificar ágilmente si un profesor tiene el permiso exacto para modificar las notas, en lugar de consultar múltiples campos en una base de datos, optimizando drásticamente la carga en los servidores de la nube.

2.8. EL ROL DEL ÁLGEBRA DE BOOLE EN EL DISEÑO DE REDES NEURONALES E INTELIGENCIA ARTIFICIAL

La Inteligencia Artificial (IA) parece magia, pero en su nivel más profundo, se asienta sobre la lógica de Boole. En las redes neuronales artificiales, un "perceptrón" (la unidad mínima de la red) actúa esencialmente como una función lógica muy compleja. Evalúa múltiples entradas pesadas y, mediante una "función de activación" (que decide un estado binario o un umbral), determina si la neurona "dispara" o no (1 o 0). Comprender a la perfección cómo se combinan estas decisiones binarias es el primer paso para entender arquitecturas complejas de Machine Learning.

CIERRE TEÓRICO OBLIGATORIO

❑ Mitos vs. Realidades

Mito Común	Realidad Tecnológica
"El álgebra de Boole es solo teoría matemática que nunca usaré al programar páginas web o configurar bases de datos."	Falso. Toda estructura condicional (if, while) en programación, filtros de bases de datos y configuraciones en la nube utilizan pura lógica booleana.
"Las compuertas lógicas son aparatos grandes de laboratorios de física; ya no existen en las computadoras modernas."	Falso. Un procesador moderno de tu celular contiene miles de millones de compuertas lógicas microscópicas integradas en el silicio.

❑ Glosario de Términos

1. **Bit:** (Binary Digit) La unidad más pequeña de información en informática, representa un 1 o un 0.
2. **Tabla de Verdad:** Representación tabular que muestra el valor de salida de un circuito lógico para cada combinación posible de variables de entrada.



3. **Compuerta Universal:** Aquella compuerta (como NAND o NOR) con la cual es posible implementar cualquier otra función lógica sin necesidad de componentes diferentes.
4. **Mapa de Karnaugh:** Diagrama utilizado para la simplificación y minimización algebraica de funciones booleanas.
5. **Microcontrolador:** Un circuito integrado programable, capaz de ejecutar órdenes grabadas en su memoria y controlar hardware externo (como relés o sensores) en base a lógica booleana.

□ **Ponte a Prueba**

1. ¿Qué compuerta lógica entrega una salida Verdadera (1) SOLO SI todas sus entradas son Verdaderas (1)?
 - a) OR
 - b) AND
 - c) XOR
2. Según el teorema de De Morgan, la expresión $\overline{A \cdot B}$ es igual a:
 - a) $A + B$
 - b) $\overline{A} \cdot \overline{B}$
 - c) $\overline{A} + \overline{B}$
3. ¿Cuál de las siguientes compuertas se conoce comúnmente como "inversor"?
 - a) NOT
 - b) NAND
 - c) OR



VALORACIÓN



Como Técnico en Sistemas Informáticos que se formará en el CEA Herman Ortiz Camargo, es crucial reflexionar sobre el impacto de nuestras decisiones lógicas. Cuando diseñamos un circuito o programamos un microcontrolador que automatiza procesos, estamos tomando responsabilidades éticas.



Pensemos en la seguridad. Si la lógica booleana que programa el acceso a una base de datos institucional en el CEA tiene un error (un OR donde debía ir un AND), podríamos exponer información confidencial de cientos de estudiantes de Pailón al acceso público. La lógica no perdona descuidos; un error binario puede significar una brecha de seguridad grave.

Además, debemos considerar el impacto medioambiental (e-waste). Diseñar circuitos optimizados usando los Teoremas de Boole o Mapas de Karnaugh significa utilizar menos componentes electrónicos, microchips más pequeños y menos energía para procesar información. Cuando un hardware queda obsoleto rápidamente por ineficiencia de diseño, termina en vertederos, contaminando con metales pesados. Al programar la automatización para el Barrio San Antonio o San José, nuestra buena lógica matemática contribuye directamente a crear tecnología más eficiente, duradera y respetuosa con nuestro ecosistema local en la provincia de Chiquitos.



PRODUCCIÓN



LABORATORIO PRÁCTICO: SIMULANDO EL SISTEMA DE SEGURIDAD DEL CEA CON PYTHON

En este reto, aplicarás la lógica booleana directamente en la computadora. Vamos a simular el sistema de seguridad que mencionamos en el "Momento 1" mediante código Python. Escribiremos un pequeño script que evalúe si la alarma del CEA Herman Ortiz Camargo debe dispararse basándose en los estados lógicos de dos sensores (puerta y ventana).

Paso 1: Abrir el entorno de desarrollo.

Abre un editor de código o una consola en línea de Python (como Replit o el IDLE de Python instalado en tu equipo).

Paso 2: Transcribir el código de simulación lógica.

Copia, analiza y ejecuta el siguiente código. Observa cómo las palabras reservadas `or` y `and` representan las compuertas lógicas directamente en el software.



Python

```
# =====
# LABORATORIO: LÓGICA BOOLEANA APLICADA - SISTEMA CEA HERMAN ORTIZ C.
# Técnico en Sistemas Informáticos - Prueba de Alarma Barrio Saó
# =====
# 1. Definimos el estado inicial de Los sensores (Entradas/Proposiciones)
# True = Abierto/Detectado (1), False = Cerrado/Seguro (0)
sensor_puerta_principal = False  sensor_ventana_trasera = True    sistema_armado
= True                          print("
--- INICIANDO DIAGNÓSTICO DEL SISTEMA EN BARRIO SAÓ -
--")
# 2. Aplicamos La Lógica Booleana
# Condición: ¿Hay alguna brecha? (Uso de compuerta OR)
# Si la puerta O la ventana están abiertas, La brecha es True (1)
brecha_detectada = sensor_puerta_principal or sensor_ventana_trasera
# 3. Decisión final de activar la alarma (Uso de compuerta AND)
# La alarma suena SOLO SI hay brecha AND el sistema está armado
brecha_detectada and sistema_armado
# 4. Mostrar el resultado por consola
print(f"
Estado de la Puerta: {
,
Abierta
' if sensor_puerta_principal else '
Cerrada'}
")
print(f"
Estado de la Ventana: {
,
Abierta
' if sensor_ventana_trasera else '
Cerrada'}
")
print(f"¿Brecha de seguridad detectada?: {
brecha_detectada}
")
print("\n
--- RESOLUCIÓN LÓGICA -
--")
if disparar_alarma: # Si evaluar_alarma == True    print("¡ALERTA CRÍTICA!
Activando sirenas en el módulo.")
else:    print("
Sistema seguro. Ninguna anomalía reportada.")
```

Paso 3: Experimentación (Tu Turno).

Cambia los valores de sensor_puerta_principal, sensor_ventana_trasera y sistema_armado a diferentes combinaciones de True y False. Ejecuta el código cada vez para construir tu propia



tabla de verdad mental y verificar si el resultado concuerda con las reglas del Álgebra de Boole estudiadas.

RECURSOS ADICIONALES

Para profundizar tus conocimientos como Técnico, se recomienda explorar la siguiente documentación oficial y recursos prácticos:

1. **MDN Web Docs - Operadores Lógicos:** Documentación oficial y didáctica sobre cómo se implementa la lógica booleana (&&, ||, !) en el desarrollo de software (JavaScript). Excelente para vincular el diseño web con este tema. Buscar en la web oficial de Mozilla Developer Network.
2. **Tinkercad Circuits de Autodesk:** Herramienta gratuita en línea que te permite construir circuitos virtuales, conectar relés y probar compuertas lógicas integradas (como la serie 7400) sin riesgo de quemar componentes físicos reales en tu aprendizaje inicial.
3. **Documentación Oficial de Python (Operaciones booleanas):** Ideal para expandir el laboratorio realizado en el Momento 4. Te enseñará cómo los lenguajes de backend manejan evaluaciones cortocircuitadas en base a reglas booleanas. Buscar en la documentación oficial (docs.python.org).